Python for computational Problem Solving · Unit_04 · UNIT 4: Testing ,Debugging and Object Oriented Programming

DRAFT — UE26CS151A · Unit_04 — UNIT 4: Testing ,Debugging and Object Oriented Programming · Complete Unit Master

Notes-first study master · source-cited lecture explanations and compact page takeaways

Hugging Face PESU_Materials source tree, official units.json catalogue, transcript sidecars, slide text/PDF/PPT sources and the selected First_Year_8enterprise archive; see the provenance datasets in the release ZIP. Generated 18 October 2026.

· How to use this pack

Read the edited lesson notes by topic. Evidence chips point to transcript intervals and slide pages. The complete source transcripts, page text, formula candidates, and image locators are preserved in sibling JSON datasets. Topics include the applicable parts: overview, key terms, formulas, figures, flowcharts, important points, worked examples, past-paper questions and a recap. IDs are shared with the separate documents: E = formula, G = graph, F = flowchart, W = worked example. Each ID is numbered by topic, so E4.2 is the second formula of topic 4.

Source text	Verbatim or OCR-derived text, kept separate from editorial notes.
Candidate	Automated topic/source association requiring manual confirmation.
Low ASR confidence	Transcript segment requires audio verification.
DRAFT	Human review remains incomplete.

Contents

1. 1 · UE26CS151A · Unit_04: UNIT 4: Testing ,Debugging and Object Oriented Programming
2. 2 · Pytest, doctest, pdb
3. 3 · Pytest, doctest, pdb
4. 4 · Python Classes and Objects
5. 5 · Inheritance
6. 6 · Exceptions
7. 7 · Source slide deck: Virtual Environments
8. 8 · Source slide deck: UE24CS151A PCPS Unit 4 SindhuRPai Class 8 9 OOP
9. 9 · Source slide deck: UE24CS151A PCPS Unit 4 SindhuRPai Class 11 Polymorphism
10. 10 · Source slide deck: UE24CS151A PCPS Unit 4 SindhuRPai Class 12 Iterators
11. Appendix A · Symbol glossary
12. Appendix B · Sources, coverage and gaps
13. Appendix C · Revision and retrieval practice

M1

Unit_04 · UNIT 4: Testing ,Debugging and Object Oriented Programming

1 UE26CS151A · Unit_04: UNIT 4: Testing ,Debugging and Object Oriented Programming

This unit master organizes source material for Python for computational Problem Solving, Unit_04, officially titled UNIT 4: Testing ,Debugging and Object Oriented Programming. It includes 5 video transcripts and 7 source slide decks, with page-level text and rendered images where available. This draft preserves source wording and page order; it does not claim a complete human subject review or full audiovisual review.
EV0000001

SHORT-NOTE RECAP

Unit map · UE26CS151A · Unit_04: UNIT 4: Testing ,Debugging and Object Oriented Programming

Official catalogue entry: UE26CS151A, Unit_04 (UNIT 4: Testing ,Debugging and Object Oriented Programming). Bucket folder present: yes. Videos: 5; slide decks: 7.

EV0000001

2 Pytest, doctest, pdb

Study focus: Pytest, doctest, pdb. This topic explains the central idea, its conditions, and how the lecture applies it in context. Review the selected source statements and follow their citations for the full lesson segment. EV0000002 · EV0000003

SHORT-NOTE RECAP

Core explanation · 00:00–04:59

Now I am going to demonstrate the value written from the normal function has to be tested in the test method of the PyTest product.

Key details

- Edit followed by the file name.

- With the file name in the test, framework has to be start with the keyword test followed by.
- If you want to pass two parameter also you can pass but I will use only one parameter.
- Then so after this I want to return the value from this function.
- We know to return the value from this function we have to use the statement written and I will give the statement as x plus one.
- So after writing the normal function now I will write the test method of the pie test.
- So how to write the test method of the pie test?
- We know we have to use the keyword test followed by the method name.
- Now to do this we know we have the statement in the PyTest framework that is asset followed by the condition.
- I will pass the value as two then I will check with the value is equal to t and I will give the statement within a double course test passed.
- See the code.
- X takes the value of 2, 2 plus 1 is 3, yes, 3 is equal to 3 is test got passed.
- 0.01 seconds.
- In this way we can check the value written from the normal function can be checked in the test method of the pie test framework.
- Then I will write the test method using B followed by test underscore method one.
- Then I will write some quote here that is I will write x is equal to 2 and I will write y equals 3 how to check the values the condition we know we have the statement I said followed by the condition I will use x not equal to y.
- A S is a set and X is equal to Y and I will write some statements in the double quotes test got passed and I will say the program and I will exit from the program.

Example or application from the lecture

- File name I will give the file name as example two dot extension should be dot p y.
- So I will keep the condition f u and c because I have used the normal function as f unc and I will pass only one value because in the function definition I have given only one parameter that's why we have to pass only one value.
- Py dot test followed by the file name where the file name is test underscore example two dot two one so this particular statement execute the program see in the output it's collected one item the test has passed because.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000002 · EV0000003 · EV0000004 · +1 source units in pack.json

Core explanation · 05:00–09:59

So we have the one more method.

Key details

- So totally two test method we have executed.
- Like this we can run n [ASR low confidence; check audio].
- I will give the directory name.
- navigate into this directory.

- Then I will write the function definition df followed by test method 1.
- Then I will write some statements like x is equal to 2 and I will write y is equal to 2.
- After this I will check the connution using the statement asset followed by X equals Y and I will write the statement state got.

Example or application from the lecture

- example 1 another is from example 2.
- Suppose if I want to test thousand files, I can't give all the thousand files in a command line like pi dot test followed by file one, file two, file three can't give.
- Suppose if I give just pi dot what will happen so it will execute all the programs that is there in your folder so now I can't make what actually it's going on what is the output of this.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000006 · EV0000007 · EV0000008 · +1 source units in pack.json

Core explanation · 10:00–14:59

that is there in that directory that is testing seeing the proof you got collected two items this indicates that you have executed the two test methods in the file test example one and test example two and both got passed in 0.01 seconds so like this you can execute n number of test methods in n number of files by giving the command pi dot test.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000010 · EV0000011 · EV0000012 · +1 source units in pack.json

3 Pytest, doctest, pdb

Study focus: Pytest, doctest, pdb. This topic explains the central idea, its conditions, and how the lecture applies it in context. Review the selected source statements and follow their citations for the full lesson segment. EV0000014 · EV0000015

SHORT-NOTE RECAP

Core explanation · 00:00–04:59

So first you create the file gedit file one.

Key details

- This particular statement will import the PDB.
- module in your program.
- That is PDB followed by dot followed by the function name that is set trace.
- If you give this statement, it will access the set trace method from the PDB module.
- In number, I am taking my number as 3, I am calling the function fact, I am going to pass the value as 3 because I want to calculate the factor of 3.
- and have to exit my program so control Q now I am going to give the command python followed by file name.py if you are using the second version give python followed by file name.py.
- If you are using the third version, you are supposed to give python 3 followed by file name.py.
- to print the value of a variable so first we need to create one program using g edit g edit followed by variable is my file name with the extension dot py.
- We know we are supposed to give the statement import followed by PDB.
- So next I'm use the variable A with the value 10.
- And how to trace we know we are supposed to give pdb dot set trace is a method that is available in the pdb.
- So this will trace each and every line what will happen in my program next I'm print my variable using the statement print and the variable is a and I will say my program control s control q is I will quit my program now I am going to execute we know how to execute the program python followed by.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000014 · EV0000015 · EV0000016 · +1 source units in pack.json

Core explanation · 05:00–09:59

variable dot p y so this will enter into my pdb debugger now if I want to print the value of the variable what is my variable have used in my program have used the variable a if I want to.

Key details

- No the value of that variable A I have to give the command P followed by variable name where the variable name in my program is A and you press enter you will get the value of that variable that is A.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000018 · EV0000019 · EV0000020 · +1 source units in pack.json

4 Python Classes and Objects

Study focus: Python Classes and Objects. This topic explains the central idea, its conditions, and how the lecture applies it in context. Review the selected source statements and follow their citations for the full lesson segment. EV0000022 · EV0000023

SHORT-NOTE RECAP

Core explanation · 00:00–04:59

Python for computational problem solving.

Key details

- Today we will try to learn how do we create a class.
- How do we work with the class?
- In order to create a class, I need to make use of a keyword called as class.
- I need to make use of a keyword called as a class.
- Followed by you will also have to specify what is the name of your particular class.
- You can give your name itself as a name of the class, but.
- it is always better to name it in a proper convention.
- The reason is whenever you are coming back in order to look at for look at your program and again, it should make sense in order to tell that what exactly you were trying to do.
- I will be telling a step by step at this point.
- Try to understand that I do not want to do anything.
- That's why I say it as a pass.
- Next, what I will be doing, nothing else, I'll just say a print.
- I'll tell what is the type of my class.
- Type of my class.
- The reason is we have said the class that you are going to be creating is a type.
- Because type specifies what you have to do, but it never specifies how you have to do.
- a small correction in the previous video I looked in the spelling of specifies was written wrong.
- Please correct it.
- What is I have said it as class itself.
- And find out.
- EX1's type is type itself.
- I need to create my own type.
- I'm trying to create my own type.
- So this is our first program.
- How do we create a class?
- We will extend this in order to make our life simple, let me comment it like this.
- Okay, so now the next point that we have to understand is a class has a behavior.
- Class has a behavior.
- And I will create a function.
- I do not want to do anything other than just writing a print statement.

- This is fine.
- Nothing else that I want to do.
- I can access it by making use of a function name.
- Dot the function name.
- Class name dot the function name.
- We said that a class has a behavior.
- Behavior in the sense it's a function.
- And if I say what is the behavior of a student, he has to listen to the class.
- What is the behavior of a teacher?
- That's the functionality.

Example or application from the lecture

- The name of my class is example one.
- We'll look with an example.
- So I will say class EX2 is what I will say because we are into the second example.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000022 · EV0000023 · EV0000024 · +1 source units in pack.json

Core explanation · 05:00–09:59

A class has a behavior and the behavior that I defined as what?

Key details

- The function f1.
- It has to be done using your ex2 dot f1.
- This I call it as invocation of a function.
- A function is getting invoked.
- So we'll go ahead with the next one.
- The next one is a class also has.
- Class also has attributes.
- Attributes in the sense fields.
- It can also be a field or it can be a variable.
- Clause, let me do it as EX3, because this is my EX3.
- Okay because this is my variable.
- Class concept something.
- Hope this is fine for you.
- I do not want to do anything else in this, so let it be as simple as possible.
- I have to access A, but it is a part of which class?
- It's a part of EX3 class.
- It says that class concept.
- maybe I did not execute the output for the previous one.
- And one thing that should always needs to be remembered is the indentation.
- It's an header.
- Important thing is the indentation.

- This is called as an header.
- The suit will follow the required indentation.
- I always use indentation, which is going to be the tab itself.
- This statement says it is out of the class.
- Anything within the class with has with some indentation.
- I say that this is a body of the class or the suit.
- S-U-I-T, we usually pronounce it as what?
- Suite of the class.
- And this is called as an header.
- Anything that ends with the colon, hope you remember that it is called as what?
- This itself becomes an header, and this becomes a body of this particular header, and so on.
- What we have done, we created a class, we created a behavior for the class, we also created an attribute for a particular class.
- We create a variable for a class which is of type of that particular class.
- What does that mean?
- See, when you when you I when I say you are a PAS student.
- You are a PES student.
- You belong to A section.
- I'll just assume it as it is sixteen students.
- How I will name each of the students.
- At this point I may say role number one, role number two, role number three, four, and so on.

Example or application from the lecture

- We will see the output that I'm expecting is what in F1 because when on this example is executed and here I need to get the output called as class concept.
- Let's move on to the next one.
- And suppose you belong to A section.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000026 · EV0000027 · EV0000028 · +1 source units in pack.json

Core explanation · 10:00–14:59

I can say that section A is a class which has so many number of variables, and these variables are of type A section.

Key details

- A section type.
- This is also called as an object.
- It is also called as an instance.
- We are going to be creating an instance.
- Do you want to do anything?
- No, I don't want to do anything at this point.
- as we are talking about A section.

- Syntax that you have to follow whenever you're trying to create an object.
- Print type of A.
- Type of A is what?
- It A belongs to your EX4 class and it is a part of your main class.
- The moment you create a class.
- Hope all the concepts are clear to you till now.
- A class, it doesn't have any implementation.
- I just created an object.
- Internally, there is a special function that gets created.
- And this special function is called as underscore, underscore, init underscore, underscore.
- You should always remember this, even though you do not need to create at this point internally, what has happened?
- Internally there is an underscore, underscore in it, underscore, underscore.
- It gets created.
- in conceptually or internally what exactly this means.
- This means that there is a function, special function gets created which has an argument A.
- We will complete this particular program.
- We will explicitly create underscore in it underscore.
- Okay, once an object is created, internally there is a method that gets executed, which is called as what?
- A special function, and we will give a name to the special function, and the name of the special function is called as constructor.
- It does a constructive job.
- Let me remove the comment and put it here.
- Let me create a class.

Example or application from the lecture

- So I will also take up an example called as A is equal to I need to make use of an EX4, which is the name of the class itself.
- Let's find out what is a type of A.
- We will try to understand with an example.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000030 · EV0000031 · EV0000032 · +1 source units in pack.json

Core explanation · 15:00–19:59

Okay, because I said a special function that gets created.

Key details

- Oh what do we print?
- We usually say it as constructor.
- Instead of typing such a big word, I just said it as construct.
- What do I do?
- I'll just create a object.

- Of type what?
- Nothing else that I really want to do here.
- And if you can see it throws me an error.
- It is throwing me an error.
- The reason is internally it looks like this, right?
- It looks like underscore, underscore of A.
- Okay, so an instance is passed as the first argument to this particular constructor.
- Let me repeat, when you do this, the instance itself is passed as a first argument.
- I need to pass something.
- Let me repeat once again, the moment I create an object, internally the constructors gets executed.
- Fine, even though this constructor has an imp implicit in nature, but what will happen is the object or an instance becomes.
- As an argument.
- It's an argument.
- There is nobody here to hold my argument.
- Because of which I have to explicitly specify a particular parameter.
- I can specify a parameter.
- There is no problem with this.
- It's absolutely working fine.
- You can give any name to the parameter, but by practice we always use something called as self.
- This is normally it we use it.
- Okay there is no role because we previously we executed with the parameter itself.
- There was no harm in executing, but why self?
- Self is generally used.
- It's a.
- It's by practice we can say.
- You use your own name, it is absolutely fine.
- And we will try to identify what we are trying to say.
- Okay because I said internally it is passed as what?
- A, which means self is A.
- A is self.
- The output that I'm expecting is both of them should print and give me the same answer.
- Okay, and if you can see both of them are pointing to the same ones, which means A is same as self, self is same as A.
- I hope there are no doubts in this case.
- Sorry if there are any spelling mistakes.
- It's not intentional.
- So define why do I need to do init method?
- It's a constructor.
- So what should go to my init method?
- The first parameter should be what?
- To hold the instance of this particular class.

- As I'm talking about rectangle, I also speak about what?
- I also speak about breadth.

Example or application from the lecture

- Let's execute.
- Let's print self here.
- This probably this is the last example that I would be taking in order to explain the concept of creation of class.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000034 · EV0000035 · EV0000036 · +1 source units in pack.json

Core explanation · 20:00–24:59

Okay, so what is the next thing that I have to do?

Key details

- Length of which one?
- What length of which one is what I'm asking?
- Of that particular instance.
- How do I point that instance?
- Self dot length okay this is equal to L.
- Fine, hop this is clear.
- I hope there are no problems here.
- I will say print lent.
- Let me do this.
- There is nothing because what happens when I execute this?
- Header is process.
- I never get into the body until we call this particular an object for this particular class gets created.
- I'm going to write the program completely.
- I what do we do?
- We just print a length and a breadth.
- Nothing much we want to do it here.
- Length into breadth is what you can say, but that we will try to do it later on.
- I'll say self.length.
- Comma self dot breadth.
- The reason why I commented also I will let at this point, please just bear with me.
- R is the object for this rectangle.
- How do I need to create an object?
- Use the name of the class.
- C T A N G L E, a big name.
- Internally, this is going to be like underscore in it underscore of R.
- But what do I need?
- I need to.

- find what is the length and what is the breadth.
- I want to print both of them.
- I will pass the values like 10 comma twenty.
- 10 comma 20.
- Okay, so ten and twenty because we are not doing anything, we're just trying to print this.
- Push it here.
- And if you can see name length is not defined as what it was trying to say.
- But when I commented this, I'm getting the output properly.
- The main reason is.
- Whenever you're making use of a variable, you should always use a fully qualified name.
- Length of which one?
- Length of this particular object.
- It should be self.length.
- Let me execute both of them.
- Length is 10, length is 10, comma 20.
- In this class, a pretty good way of we have understood so many things.
- We created an empty class, we came to know that class has a behavior, class has an attributes, okay.
- In this case, we came to know that there is an.
- Invalid function that gets invoked on creation of a particular object, which is called as a constructor.
- We have used constructor in order to initialize the variables.
- So in the next class, what we will do is we will try to find out how I can add.

Example or application from the lecture

- Let's quickly just execute this to find out what is going to happen.
- So now this is an example where a particular class has a function, but this is a built-in function.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000038 · EV0000039 · EV0000040 · +1 source units in pack.json

Core explanation · 25:00–29:59

function to this particular class.

Key details

- Because we have done the function separately and we have done the attributes separately, but we will try to find out how do we add a function to this particular class.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000042 · EV0000043 · EV0000044 · +1 source units in pack.json

5 Inheritance

Study focus: Inheritance. This topic explains the central idea, its conditions, and how the lecture applies it in context. Review the selected source statements and follow their citations for the full lesson segment. EV0000046 · EV0000047

SHORT-NOTE RECAP

Core explanation · 00:00–04:59

In the last class, we looked into the concept of encapsulation.

Key details

- we are going to be learning the concept of inheritance.
- When I say I say a cat is a mammal, tiger is a cat, lion is also a cat.
- Animals, they have a common characteristic.
- They walk on their toes.
- There may be they climb the tree.
- There may be different ways in which they walk or they try to climb, but still all of them we address them as what?
- The cat itself.
- If an easy relationship that exists, fine, then we call it as inheritance.
- A cat is a mammal, tiger is a cat, lion is a cat, and so on.
- That is also called as an inheritance.
- You inherit properties from your ancestors.
- And now do I really want to do anything in this case?
- This is what I will do.
- What I really want to do here, nothing great other than just a print here.
- But I said I want class B to inherit the properties of class A.
- I want to inherit the properties of class A.
- So which means B is a class which inherits from A.
- So which means class B inherits from A.
- So even though I try to print only one statement because it is inheriting, so I will also get the information that is present in your class A.
- And then let me execute.
- I'll just do a simple modification to this program and I'll say A is equal to twenty.
- I don't need a print statement.
- We have created.
- An object for a class B, but I know that class B inherits from class A.
- Because of the inheritance, what should happen?
- I should also get the value called as.

Example or application from the lecture

- So we'll see an example in order to understand what exactly is inheritance, and how do we put the concept of inheritance using Python.

- The example that I would like to take is cats, tiger.
- So now why this particular example?

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000046 · EV0000047 · EV0000048 · +1 source units in pack.json

Core explanation · 05:00–09:59

Should I be here.

Key details

- See, I'll just write the value.
- Value of A is.
- And C the value of A is twenty.
- I have not created a variable A, but because of the inheritance, I am able to access, I am able to access the value because I inherit I in the sense you can.
- Class B inherits the properties from class A.
- And that's how we have got this is in class B.
- This is called as the way in which we have written B is inheriting from A.
- Single inheritance or single level inheritance.
- If you are inheriting from a single class itself, then it is called as what?
- Quickly I will code it so that you try to understand.
- I'll say A is equal to 10, B is equal to 20.
- I hope why the usage of self.
- The value sum.
- Say my class.
- inherits from operations.
- Just look what is going to happen.
- I'll say self dot B.
- What is the value of B?
- I'll see print.
- Subvalue is okay, sub itself.
- Create an object.
- Object OB is equal to for which class we are going to be doing?
- We are going to be doing it for my class.
- Please remember, my class inherits from.
- A and B is defined in class A, but as I'm inheriting, I should also get the values from the class operations.
- I'll say it as obsub.
- The moment I do this, you have created an object for your my class.
- So now I'll go to my class where my sub function is defined.
- I will try to perform what?
- From the class operation, because I am performing inheritance.
- The sub subtracted value is ten because twenty minus ten.
- Ob Please remember you are trying to access.

- The function add, which is in your operation class, but you have created an object for your my class.
- Because it is inheritance, I inherit all the properties from.

Example or application from the lecture

- Let's create it to you so that it is easy for you to understand.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000050 · EV0000051 · EV0000052 · +1 source units in pack.json

Core explanation · 10:00–14:59

The operations class, because of which variables are available, I can also access the function.

Key details

- It is single inheritance.
- Multi-level inheritance.
- Quickly do this.
- Why don't I copy paste the same thing?
- I'll create one more class.
- Oh what do we do?
- Multiplication, shall we do?
- Define a multiply.
- or yes result is equal to self dot a into self dot b self dot b please remember.
- Quickly do this and then tell you few tips for you to remember.
- final result or something.
- Final answer is what result.
- Multiple class, multi-class.
- This is fine.
- Multi inherits from my class, my class inherits from operation.
- And class B is getting inherited from class A.
- There are more than.
- One level in this case, so it is called as what?
- Multi-level inheritance.
- Sub value is thirty.
- wait a second, wait a second.
- Let me comment this, otherwise we will get more number of outputs in the final app.
- Let me do this.
- I've commented, so these are the things that we had.
- I'm not confusing.
- You can see sum is 30, subtracted value is 10, and the final answer, which is multiplication, is 200.
- Multi-level inheritance.

Example or application from the lecture

- Let's quickly go ahead and do multi-level inheritance.
- Let's quickly execute this and check.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000054 · EV0000055 · EV0000056 · +1 source units in pack.json

Core explanation · 15:00–19:59

I kindly request you to try out a program of this particular sort.

Key details

- Only change that you are going to be doing is fine.
- Instead of just my class, you will bring out this particular operations here itself.
- Please remember inheritance is happening only between class C, A, and B, and B and A are nowhere related to each other through the inheritance.
- Please always remember.
- I urge you people to kindly try out these things.
- In case of any clarifications we can always discuss.
- Hope you have understood the concept.

Example or application from the lecture

- Okay, in this particular example.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000058 · EV0000059 · EV0000060 · +1 source units in pack.json

6 Exceptions

Study focus: Exceptions. This topic explains the central idea, its conditions, and how the lecture applies it in context. Review the selected source statements and follow their citations for the full lesson segment. EV0000062 · EV0000063

SHORT-NOTE RECAP

Core explanation · 00:00–04:59

We will see one more small example.

Key details

- Unless you pay attention.
- Can see here I have created a object called.
- Taking the fruit name and the count of the fruits.
- Or a static attribute?
- Is this count?

- I have two count.
- So this count just belongs to that one particular object that we have created because it is self.count.
- But when we look here.
- I am saying fruits.com.
- We know that fruits is nothing but my class name.
- root and I have template of sum of total number of roots.
- It will reflect basically when I try to access this roots.com, obviously I'll get the simulative so apples i'll get apple count as three and food count as three where i make another object piece you can see that it is name of the food count as four now food star count should be three plus four seven is it clear any doubts there.
- When I say peach dot count, I'll get it as four.
- When I say fruits.count, we will get it as seven.
- Four plus three seven because here you can see I made a cumulative stuff.
- If at all I was only thinking of a local variable that is non-static variables, I cannot do cumulative sum like this because it does not carry the same value across all the objects.
- So here I can do it because this is a static variable.
- The apples is an object.
- I'm trying to access the class name of my object apples.
- This is another syntax to access your class variable to the static variable, and you can see a type of A.
- What do I get say if I take a variable A equal to 10 and I check for type of A?
- What will be type of A?
- It would be.
- Class is a user defined type, right?
- It will be.
- It's fruits, right?
- In our case, it's fruits.
- Apples classes foods of similarly type software.com any doubts.

Example or application from the lecture

- So this is a typical example where you can make out difference between a static variable and a normal variable.
- Is it clear you need to be familiar with all possible syntaxes because when you will be given questions to find out or error, if you don't know these syntaxes, then you'll end up not writing it properly.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000062 · EV0000063 · EV0000064 · +1 source units in pack.json

Core explanation · 05:00–09:59

Comment that operator overloading.

Key details

- Apples.count three, apple peach.com four, fruits.count seven, and then apples.class.count seven, and begin type of peaks.count seven.
- Any doubts or any clarification?
- Take down a question quickly.
- create a class create a class marks create a class calculate marks.
- In which in which a student's in which a student's individual subject marks.
- For ISA 1, ISA 2 and ESA.
- Each individual subjects, ISA one, ISA2 and ESA Marks is initialized.
- [ASR low confidence; check audio].
- [ASR low confidence; check audio].
- ISA 1, ISA 2, and ISA marks for.
- Accept ISA 1, ISA 2, and ISA marks for.
- Chemistry Python Max.
- Civil and electronics and electronics find the total marks score find the total marks score and.
- Also find and also find percentage and percentage.
- Did all of you get an idea why will you be doing this?
- You will create a class called as calculate marks, right?
- And in your init function you will be taking ISA one, ISA2, and ESA Mark.
- How many objects?
- How many objects will you get to get?
- How many subjects have I taught?
- I'll not write the entire code that you people write just to tell how you should be using a class variable only that I'll tell.
- This is your class.
- I will just say should be taking [ASR low confidence; check audio].
- You make respective assignments, right?
- Or five subjects.
- Yeah, she looks like this.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000066 · EV0000067 · EV0000068 · +1 source units in pack.json

Core explanation · 10:00–14:59

Every time say I'll get ISA one, Isa 2, and ESA for math, similarly to Python, etc.

Key details

- And then in my can I write what should I write to access total marks here?
- [ASR low confidence; check audio].
- All these right? self.isa1 plus self.isa2, etc.
- [ASR low confidence; check audio].
- Yes, under the internet country.
- No, but still you need to take self.
- Because this is with respect to math object, right?

- No, you need to be whatever I write here though that if at all this I discussed when I did the first program if at all I you might take like this dot.
- A1 is equal to IC1, then it is self dot A1.
- It is not ISA1.
- These are your variables.
- Any clarifications to complete this?
- We'll go to static methods now.
- Basically executing for which particular object.
- That's why my I pass a self here, right?
- I need to make out that accordingly it will execute so we know that this is with respect to objects, so it will be for respect to objects only.
- But when you're looking at these static methods, these are the common methods across any objects.
- Like how we are discussed every method inside a class will have one parameter by default that is cell that will take name of the object, right?
- With respect to both static method and static variables, I have done this.
- Why because I had to tell you all dictionary.
- If at all I say self-taught dictionary, what happens is that all its variables become keys and whatever value I've assigned will become value.
- So should this dictionary look like it should be name because name is my variable name colon whatever I have passed ram here, right?
- Why because with respect to each object every time I'm.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000070 · EV0000071 · EV0000072 · +1 source units in pack.json

Core explanation · 15:00–19:59

equal to one so count will be equal to one now student dot count is a class variable or the static variable so it will count number of students okay as given application like accept details of a student.

Key details

- One after the other and also get me the total count of the students who have registered.
- If this is the case, then this give me total number of students.
- So now you can see I have called the static method display count.
- Okay, I have called the static method display count.
- When I go here, it basically accesses whatever is necessary for the student document.
- Okay, and you can see here this one that is at the rate of static method, it's only optional, but this is optional when you are actually calling your method like this but if you want to call your method with respect to object then you need to be giving this syntax that is the decorative syntax any doubts any doubt is it clear.
- And this is with respect to.
- Please do not write your assignments in the class.
- Any other subject assignments, if at all you are writing, please do not write in the class.

- If at all if somebody is writing, please do not write.
- Any clarifications, any doubts?
- My everyone are looking very dull.
- You had a lot of work.
- If you don't write it's okay, anyway I'll be sending this document.
- Just see that you understand.
- This is a simple program where I've used the static method and I'm finding if a person has a second name or not.
- All of this logic, right?
- Basically, when I get a string, I can split based on spaces.
- string complete string, right?
- Scott will be one string and robin will be second string.
- Just remember this.
- Remember this logic.
- And the last topic before we end, take down composition.
- You have seen inheritance, right?

Example or application from the lecture

- I just gave as an extra example, that's all.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000074 · EV0000075 · EV0000076 · +1 source units in pack.json

Core explanation · 20:00–24:59

The parameter A that means to say that B is a derived class and A is a space class.

Key details

- Here it is something like a class as another class within itself.
- I have two independent classes.
- A and then say I have class B.
- If class B creates an object of class A, that means that object can actually access all the characteristics of my class A.
- You can see here I have a class called as component.
- And I have a method called as M1.
- All simple methods.
- I've just written the phrases.
- Nothing logical in this.
- What is that it makes it as an object composition specifically.
- I'm saying self dot object one is equal to component.
- That means to say that I am creating an object of one class inside another class.
- It composes the characteristics of some other class.
- Because I have created an object of that class inside.
- Okay, this is possible because I had created an object obtain of type component.
- Any doubts, people?

- It says component class object created that happier self.object one.
- And then I basically make HBJ2.
- So m2 method is called, and then inside my m2, I have called m1 method, so m1 method is called actually m1 is a method that belongs to my other class.
- You want to write to understand?
- Or if it's okay the next topic.
- I'll just say this, you can analyze once.
- Difference between inheritance and composition is that in inheritance you will not create any object of the other class, but you will need that inherited as a parameter.
- You have not inherited, but you have created object of another class.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000078 · EV0000079 · EV0000080 · +1 source units in pack.json

Core explanation · 25:00–29:59

S students are present now.

Key details

- It's Vincent, Aditya, Krishna, Anshal, Abai, R, Abhishek, yes.
- Abhishek is Kupri.
- You were present.
- Morning you were absent.
- Please remove your mask when you present morning.
- You can just recollect.
- Objects are basically those variables which will allow you to access the characteristic features of the class, or you can also say object is an instance of a class, and then functions inside a class by default it will have variable parameter which is called a self unless and until it is a static method okay you studied about constructors you studied about parametrized and non-parameterized constructors.
- And then you studied about these default functions which are automatically accessed when you pass this can be access when you pass a string to another function, right?
- And then you studied about access modifiers, public and private, right?
- By default, what are the type of methods and variables inside a class?
- It is private All of you are so colour.
- It will be what public or private?
- By default it will be.
- Otherwise if it needs to be private, then you need to prefix the underscore, underscore.
- Then you studied about inheritance.
- You studied about multiple inheritance.
- And then you studied out to call in it function, huh?
- Base class, from base class, I mean from derived class, you also call base classes in its function, and you also studied about super method, which basically allows you to access methods and in it method from your base class.

- You studied about static variables, static methods, operator overloading, and then you studied about composition and you also studied about.
- Function overloading, right?
- Or method overloading.
- Remember all these concepts.
- Any clarification among all these topics?
- Is it clear?
- Is it clear to all of you?
- Hope you have been doing your own work problem that I'm giving it take only 10 to 15 minutes.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000082 · EV0000083 · EV0000084 · +1 source units in pack.json

Core explanation · 30:00–34:59

Take down the next topic exception handling.

Key details

- As the name says when we say exception, always it'll be normal except for certain reasons it might not work.
- Something like in your timetable.
- Whichever subjects are there, those subjects will be taught on those respective days unless and until the faculty is absent.
- What are more errors we have?
- We have syntax errors, we have semantic errors.
- We don't have any problem with syntax errors, because your interpreter is capable of handling it.
- When it comes to your semantic error, we will have to deal with it.
- We saw by test and we saw it basically tells us where exactly we have gone wrong.
- One more way is your exceptions.
- It basically tells us that how we need to handle those semantic errors.
- Something like when I try to divide a number by zero.
- And in that exception, all these inbuilt exceptions are being defined.
- Something like divide by zero, index out of form.
- And I try to enter something about two hundred that cannot be anticipated by your interpreter, right?
- Because that is application oriented.
- My application maximum marks is two hundred.
- it need not be same always so such application dependent semantic errors should be handled by the programmer itself okay by writing a user defined exception is that clear so.
- Basically, when I when we talk about exception handling, we will think about the errors.
- This is shown by my interpreter itself.
- If I take AB and I say C equal to AB to give me a error, right?
- I told about written exceptions and user defined exceptions.

- It's not necessary that I should be using the same message.
- I can also override on this like how we did overriding of underscore add like that you can also overwrite these exceptions, but in bed I have certain phrase about these exceptions.
- Zero division error.
- I'm trying to divide by zero.
- Then we have IO error.
- Then I told about index error.
- I don't have a variable called as byte.
- But still I try to print my that means that it's a name error because I don't have a variable of that.
- And we have a type error.
- I'm trying to add a string and a number, right?
- I cannot do that because there's a type error.
- I cannot add a string and a picture.
- And we have something called as value error.

Example or application from the lecture

- And say for example I have a list, I have five elements in the list, I'm trying to access the sixth element in the list.
- All these exceptions have been given.
- But say for example, I'm basically taking your map and maximum marks is two hundred.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000086 · EV0000087 · EV0000088 · +1 source units in pack.json

Core explanation · 35:00–39:59

So we'll see if you have the example executed here.

Key details

- I have x equal to 10, it's an interactive mode, and I try to print y.
- Okay, so you can see here I have a name error because Y is not defined.
- Here I'm taking a number and this is also a number but it's in form of a string, right?
- Because type mismatch.
- Here I have index.
- There's no where number call is 40 in my list, but I'm trying to find an index where 40 resides.
- I'm trying to access some value which is not there, so it's a value error.
- And if I'm trying to say in this case itself, I try to access say L of four.
- I have zero one two index.
- If I try to access L of 4 index, fourth index is not there, so it will be index out of bounds exception.
- These are the common things that you need to remember.
- Then I have a list try to access this.
- I've got axe of the form.

- And then A plus 10, it's a name error because I don't have a variable called A and I'm trying to increment it by 10.
- Name error, tenure, two and six.
- I cannot multiply two strings, so it's the type error.
- Here I cannot convert this floor which is in form of a string to an integer.
- I cannot type cast this, right?
- These are all the exceptions.
- I'm just going fast because it's simple and nobody's asking me anything.
- Simple, what we will do is that I have a program.
- You will we can use we will use a keyword called as except.
- Which basically this block of code which is under accept will get executed only if an exception is raised in the right block.
- I try to say it's A divided by B, I accept both values of A and B.
- And if it is say A is 100 and B is 20, so there's no exception, right?
- But when I give value of A as 10 and value of B as zero, then there will be an exception at that point of time.
- Set up instruction which is under it, okay?
- [ASR low confidence; check audio].
- We will just see what happens if I just executed it as it is.
- This is what we get.

Example or application from the lecture

- A simple example, file print x, an exception occurred as it is, not the fight.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000090 · EV0000091 · EV0000092 · +1 source units in pack.json

Core explanation · 40:00–44:59

When I did not use except, this is your error is shown by your interpreter.

Key details

- Obviously, if you are writing a application for your client, he would not like to see this.
- Because he doesn't understand what this is.
- Unless Until he is a programmer, he will not understand what this usually clients say it will be non-programmer, right?
- This is not handle, this is.
- No, this is here I had handled it like this, a tri block and a accept block.
- I can also handle it by giving our.
- Inbuilt exception name.
- Say for divide by zero, I have inbuilt exception.
- This is how we can add it by using [ASR low confidence; check audio].
- Inbuilt exception, this is how we end.
- Here again we have handled the name error.
- I'm dividing it by zero.

- Okay, so basically it is giving me the error because it's not a name error, right?
- Actually y equal to one divided by zero was a zero division error.
- But while writing my except block, I wrote I am.
- Use a wrong inbuilt exception.
- Say my trial block has some 10 instructions.
- In that I can accept that could be a divide by zero, that could be an index of, it could be something else as well.
- I can have any number of except blocks.
- For a single tribal, you can have any number of except blocks.
- In this case, when an exception was raised, so divide by zero error was raised.
- And then you can see out of.
- Try accept block.
- It doesn't mean that there's an exception, it will not execute the rest of the code, it will still execute the rest of the code.
- Can you doubt?
- So it will first look for the first match.
- Say something like there are set up except block, so it will check the exception which has got raised.
- Does it belong to this category?
- It doesn't, then it goes to next category.
- And then it goes to next category.
- Like you have two similar kinds of exceptions which is occurred.
- In that case, it will take the first exception block which is there.
- It will not look for the best exception block.

Example or application from the lecture

- How does it basically consider which except block to be executed when there is some exception?

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000094 · EV0000095 · EV0000096 · +1 source units in pack.json

Core explanation · 45:00–49:59

It will look for the first matching except not.

Key details

- Here A is 5 and B is 6.
- With except you can have a else block.
- One pride block can have any number of except block.
- Basically, this is when no error occurs.
- Earlier, what used to happen?
- If the error occurs, it goes to next step block.
- But if error does not occur, you want set of operations to be done, then you can put it under else block.

- Is that clear?
- When do we use the else block?
- If at all you have two things to decide if a error, if an exception occurs, what to do.
- And if exception does not occur, what to do, then what to do if an exception does not occur, you can put it under else block.
- This is basically raising our own exceptions.
- Till now you have seen all built-in exceptions.
- I'm trying to enter marks with greater than 200.
- That is not input exception, so I have raise my own exception.
- Please take syntax of this.
- Take syntax of your user defined exception.
- Abhishek, what are you doing?
- I don't see.
- I hope you have taken the normal inbuilt exception handling, right?
- Try and then except.
- You can try it any number of except.
- [ASR low confidence; check audio].
- And set off except with else, yeah.
- Can you just remove your mask?
- [ASR low confidence; check audio].
- At what time?

Example or application from the lecture

- Say, for example, the example that I gave, Marks is maximum marks is 200.

These are edited study notes based on the cited lecture interval. Check uncertain speech recognition against the source transcript and recording.

EV0000098 · EV0000099 · EV0000100 · +1 source units in pack.json

7 Source slide deck: Virtual Environments

Study focus: Source slide deck: Virtual Environments. The notes identify each page heading, summarize its main ideas, and explain its place in this unit; citations preserve the source page locator. EV0000186 · EV0000187

SHORT-NOTE RECAP

Study notes · Source slide deck: Virtual Environments

The slide deck is rewritten here as searchable lesson notes. Each subsection states the slide heading and its concise teaching points; page citations lead back to the indexed source image when visual details are needed.

Slide 1 · Solving- UE26CS151A

- Image-only source page; see the visual page locator in `datasets/content-index.json`.

Slide 2 · Why Do We Need Python Environments?

- Different projects may require different Python or package
- Virtual environments isolate project dependencies from one
- system Python and simplifies deployment

Slide 3 · Introduction to Python Environments

- A Python environment is a workspace where Python and its
- Separate environments help avoid conflicts between package
- Key topics covered: virtual environments, venv, pip and

Slide 4 · What is a Virtual Environment?

- Python provides the built-in venv module for creating virtual
- No extra installation needed — venv ships with Python 3.3 and later
- Each environment behaves like its own self-contained Python installation

Slide 5 · Creating a Virtual Environment

- Navigate to the project directory first
- Create an environment using venv:
- Environment names such as myenv, venv or .venv are common

Slide 6 · Activating a Virtual Environment

- An active environment is shown by (myenv) in the terminal prompt

Slide 7 · Deactivating a Virtual Environment

- Use deactivate when finished working with the environment
- Deactivation does not delete the environment — it simply exits it
- The same environment can be reactivated anytime later

Slide 8 · What is pip?

- pip is Python's package installer
- python -m pip install numpy
- Works both inside a virtual environment and with the system

Slide 9 · Installing Packages

- Install one package or several packages together
- Packages can be imported into Python right after installation
- python -m pip install numpy==1.26.4

Slide 10 · Upgrading, Uninstalling & Viewing Packages

- python -m pip install --upgrade numpy
- python -m pip uninstall numpy

- Package details and dependency list:

Slide 11 · requirements.txt

- python -m pip freeze > requirements.txt
- python -m pip install -r requirements.txt
- Typical project layout: program.py, requirements.txt,

Slide 12 · Sharing a Python Project

- Share the project folder along with requirements.txt — not the
- Another user creates a fresh virtual environment on their own
- They install every dependency from requirements.txt in one

Slide 13 · Complete Workflow & Example: NumPy Project

- Create project → create venv → activate → install packages →
- Example: setting up a NumPy project
- python -m pip install numpy

Slide 14 · Why Use python -m pip?

- Runs pip using the specifically selected Python interpreter
- Reduces confusion when multiple Python installations or
- python -m pip install package_name

Slide 15 · Virtual Environment vs System Python

- System Python: packages are shared across many projects, so
- Virtual environment: packages are isolated per project,
- Changes to system Python can affect the whole machine; venv

Slide 16 · Quick Reference – Important Commands

- python -m pip install numpy | python -m pip
- python -m pip freeze > requirements.txt |
- python -m pip install -r requirements.txt

Slide 17 · Key Takeaways

- Virtual environment = isolated workspace for a Python project
- pip = tool used to install and manage Python packages
- Using virtual environments makes Python development cleaner,

Slide 18 · Department of CSE (AIML)

- Dr Manjula K- manjulak@pes.edu
- Ack: Teaching Assistant - JANAVI AGRAWAL (CSE

EV0000186 · EV0000187 · EV0000188 · +51 source units in pack.json

8 Source slide deck: UE24CS151A PCPS Unit 4 SindhuRPai Class 8 9 OOP

Study focus: Source slide deck: UE24CS151A PCPS Unit 4 SindhuRPai Class 8 9 OOP. The notes identify each page heading, summarize its main ideas, and explain its place in this unit; citations preserve the source page locator. EV0000240 · EV0000241

SHORT-NOTE RECAP

Study notes · Source slide deck: UE24CS151A PCPS Unit 4 SindhuRPai Class 8 9 OOP

The slide deck is rewritten here as searchable lesson notes. Each subsection states the slide heading and its concise teaching points; page citations lead back to the indexed source image when visual details are needed.

Slide 1 · PROBLEM SOLVING

- Department of Computer Science and Engineering

Slide 2 · Object oriented Programming

- Programming Paradigm: Style or way or strategy using which we write the solution
- Procedure is a set of instructions used to accomplish a specific task.
- It can be routines, sub-routines, functions etc.

Slide 3 · Object oriented Programming

- Focus is on the data and the operations that manipulate the data.
- HelpsHelps inin organizingorganizing andand designingdesigning codecode byby representingrepresenting real-worldreal world entitiesentities asas
- OOP is mainly useful to develop big and complex projects carried out by large

Slide 4 · Object oriented Programming

- The way you view an object(Ex: employee, library, customers etc)
- Depending on the application, abstraction has to be implemented.
- Binding of data and procedures as a single unit.

Slide 5 · Object oriented Programming

- Capability of one class (child) to derive the capabilities of another class(parent)
- Code reusability is the major benefit of inheritance.
- Capability of an object of a class to behave differently in response to the data.

Slide 6 · Object oriented Programming

- It is a methodology to create an entity.
- Defines attributes (variables) and methods (functions) that all objects of that class

- A class specifies the set of instance variables and methods that are “bundled

Slide 7 · Object oriented Programming

- Syntax: (To create a class)
- <class '__main__.Ex1'> <class 'type’>
- So, Ex1 is a type in the package __main__ 7

Slide 8 · Object oriented Programming

- Example 2: class with attributes (fields or variables)
- Example 3: class with methods (behaviour)

Slide 9 · Object oriented Programming

- Example 4: Class having data and a method.
- print("This is a Petrol Car")
- This is a Petrol Car

Slide 10 · Object oriented Programming

- It is a a physical instance of a class.
- Have theirh own attributesb ((classl variables/instancebl / variables)bl) andd
- methods(class functions), as defined by the class.

Slide 11 · Object oriented Programming

- 1.Identity: Each object has a unique identity throughout its lifetime.
- The id() function can be used to get the identity of an object.
- 2.Type: The type() function can be used to determine the type of an object.

Slide 12 · Object oriented Programming

- <__main__.Car object at 0x000001C9E2200DC0>

Slide 13 · Object oriented Programming

- The existence of a class does not create the object.
- Object must be created explicitly
- Ex: c1= Car() will create an object(c1) of class Car

Slide 14 · Object oriented Programming

- It is a special function of the class which is called when an object is created
- It is invoked automatically and implicitly when the object of the class is created.
- The constructor can be used to initialize the internal state of an object, and create

Slide 15 · Object oriented Programming

- up actions when an object is destroyed or deleted.

- The destructor is automatically called just before an object is removed from
- It is often used to release resources before an object is removed from the system.

Slide 16 · Object oriented Programming

- self : <__main__.Car object at 0x000001F214B30DC0>
- c1 : <__main__.Car object at 0x000001F214B30DC0> #Same output
- attributes and methods within that instance and should be the first parameter.

Slide 17 · Object oriented Programming

- File "C:/Users/ADMIN/Desktop/practice programs.py", line 38, in
- <module> car() TypeError: __init__() takes 0 positional arguments but 1
- As we see in the output, the constructor needs a parameter called self

Slide 18 · Object oriented Programming

- Example 8: Use of constructor and destructor
- obj1 = MyClass("Object 1") # Creating objects
- del obj1 # Explicitly deleting an object

Slide 19 · Object oriented Programming

- Constructor called. Object 1 created.
- Note: The destructor is called when the object obj1 is explicitly deleted
- destructors when objects are not needed.

Slide 20 · Object oriented Programming

- ExampleExample 9:9: Non-parameterizedNon parameterized constructorsconstructors
- #when no parameters are passed for invoking constructor

Slide 21 · Object oriented Programming

- print(self.name,self.age) #all names must be fully qualified
- p = Person("joe",30) # value of self will be implicitly assigned
- p.gender=‘M’ # add instance variables to the object outside the class

Slide 22 · Object oriented Programming(Practice Program)

- Example 11:Demonstration class & object with constructor and destructor.
- def __init__(self, length, width):
- print(f"Rectangle dimensions: {self.length} x {self.width}")

Slide 23 · Object oriented Programming

- Used to retrieve the value of attribute of a class without directly exposing it.
- Used to modify the value of attribute of a class.
- Allows controlled modification of the attribute's value by performing checks or

Slide 24 · Object oriented Programming

- Ex: Getter and setter using user defined functions
- print("Name:: before calling setter",obj.get_name()) # Output: Arun
- print("Name:: after calling setter",obj.get_name()) # Output: Ram

Slide 25 · Object oriented Programming

- Syntax setattr(object, attribute, value)
- Syntax: getattr(object, attribute, default)
- 3 hasattr() function returns True if the specified object has the specified

Slide 26 · Object oriented Programming

- Ex: Getter setter using predefined functions

Slide 27 · Object oriented Programming

- 1.Define a class called “candidate” which has the Registration number and
- WriteWrite aa functionfunction Input()Input() whichwhich allowsallows thethe useruser toto enterenter valuesvalues forfor thethe aboveabove
- If score>=60 then assign remarks=“Selected” else remarks= “Not selected”

Slide 28 · Object oriented Programming

- 2. Define a class called Travel with the following descriptions:
- Define a constructor which initializes the above class members with the values 105,
- Also assign the number of buses equal to 1 if number of travelers is greater or equal

Slide 29 · Object oriented Programming

- If quantity<=100 then discount=0
- If quantity <=150 then discount is 5%
- If quantity >150 then discount is 10%

Slide 30 · Object oriented Programming

- 4.Define a class to represent the bank account of a customer with the
- information like Name of the depositor, Account number and type of the
- 3. To withdraw the amount after checking the balance amount

Slide 31 · Departmentp of Computerp Science and Engineeringg g

- Dr. Shylaja S S, Director, CDSAML & CCBD, PESU
- Prof. Sindhu R Pai – sindhurpai@pes.edu

EV0000240 · EV0000241 · EV0000242 · +90 source units in pack.json

9 Source slide deck: UE24CS151A PCPS Unit 4 SindhuRPai Class 11 Polymorphism

Study focus: Source slide deck: UE24CS151A PCPS Unit 4 SindhuRPai Class 11 Polymorphism. The notes identify each page heading, summarize its main ideas, and explain its place in this unit; citations preserve the source page locator. EV0000411 · EV0000412

SHORT-NOTE RECAP

Study notes · Source slide deck: UE24CS151A PCPS Unit 4 SindhuRPai Class 11 Polymorphism

The slide deck is rewritten here as searchable lesson notes. Each subsection states the slide heading and its concise teaching points; page citations lead back to the indexed source image when visual details are needed.

Slide 1 · PROBLEM SOLVING

- Department of Computer Science and Engineering

Slide 2 · Polymorphism refers to having multiple forms.

- refersf to the use off the same ffunction name, but with differentff signatures
- allows objects of different classes to be treated as objects of a common
- This concept enables a single interface to be used for entities of different types.

Slide 3 · Runtime Polymorphism

- Python supports runtime polymorphism by using techniques like method
- It is also known as dynamic polymorphism
- It enables the same method name to behave differently based on

Slide 4 · Key Aspects of Runtime Polymorphism:

- Runtime polymorphism is closely associated with inheritance.
- Subclasses inherit methods from their superclass, and they can
- The method signature remains the same in both the superclass and

Slide 5 · 3.Dynamic Binding:

- During runtime, the appropriate method to execute is dynamically determined
- It is based on the actual type of the object invoking the method
- Exhibits different behaviors based on their specific implementations.

Slide 6 · The following example (Refer next slide) demonstrates all these key features of

- In this example ‘Rectangle ‘ and ‘circle’ subclasses override the ‘calculate_area’

- The method ‘calculate_area’ is called on objects of different types(‘rect’ or ‘circle’) .
- The appropriate overridden method is executed based on the object’s actual type.

Slide 7 · class Shape:

- ddeff __init__(self,(lf llength,h width):d h)
- return self.length * self.width

Slide 8 · class Circle(Shape):

- return 3.14 ** self.radiuslf d ** self.radiuslf d

Slide 9 · Practice Program

- ‘AudioFile’. Both classes should have a ‘play’ method but provide
- Create a base class ‘Vehicle’ with a method ‘calculate_speed’.
- ‘calculate_speed’ method in each subclass to calculate their

Slide 10 · Departmentp of Computerp Science and Engineeringg g

- Dr. Shylaja S S, Director, CDSAML & CCBD, PESU
- Prof. Sindhu R Pai – sindhurpai@pes.edu

EV0000411 · EV0000412 · EV0000413 · +27 source units in pack.json

10 Source slide deck: UE24CS151A PCPS Unit 4 SindhuRPai Class 12 Iterators

Study focus: Source slide deck: UE24CS151A PCPS Unit 4 SindhuRPai Class 12 Iterators. The notes identify each page heading, summarize its main ideas, and explain its place in this unit; citations preserve the source page locator. EV0000441 · EV0000442

SHORT-NOTE RECAP

Study notes · Source slide deck: UE24CS151A PCPS Unit 4 SindhuRPai Class 12 Iterators

The slide deck is rewritten here as searchable lesson notes. Each subsection states the slide heading and its concise teaching points; page citations lead back to the indexed source image when visual details are needed.

Slide 1 · PROBLEM SOLVING

- Department of Computer Science and Engineering

Slide 2 · An iterator is an object that allows iteration through a sequence of

- elements, one at a time. It implements two main methods: __iter__() and
- __next__():: returns the next item in the sequence.

- When there are no more elements to return, it raises the StopIteration

Slide 3 · Iterators lazy object or Eager object?

- Lazyy Evaluation: Iterators follow the principlep p of lazyy evaluation,, meaningg
- they generate values on-demand rather than computing all values at once.
- This can be memory-efficient when dealing with large datasets as it only

Slide 4 · Custom Iterable Objects

- 1. __iter__(callable as iter(container-object)) which returns an object of a
- 2. __next__(callable(ll bl as next(iterator_object))(b))
- boolean condition – like elements greater than 100

Slide 5 · Example 1:Creates an object of MyContainer whose attribute mylist refers to

- if self.i <= len(self.mylist):
- return self.mylist[self.i - 1]

Slide 6 · a = ['apple', 'banana', 'orange', 'dates', 'cherry']

- Here, observe that it creates an object of MyContainer whose attribute
- mylist refers to the list a

Slide 7 · Working of iterators

- The for statement keeps calling next on this iterable object.
- The __next__ function has the logic to return the next element from the list
- and update the position and also raise the exception stop iteration when the

Slide 8 · Example 2

- squares of numbers from 0 to n-1 def __next__(self):
- It has __iter__() and __next__() methods if self.current >= self.n:
- When instance of this class in a for loop, it iterates square = self.current ** 2

Slide 9 · Departmentp of Computerp Science and Engineeringg g

- Dr. Shylaja S S, Director, CDSAML & CCBD, PESU
- Prof. Sindhu R Pai – sindhurpai@pes.edu

EV0000441 · EV0000442 · EV0000443 · +24 source units in pack.json

A Symbol glossary

Symbol	Name	Meaning	Appears in

B Sources, coverage and gaps

Source	Path	Use
Official unit catalogue	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/units.json	Official course/unit title and index.
Pytest, doctest, pdb transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/AV_Summaries/001_Pytest_doctest_pdb__9c2a04e725b61a85.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Pytest, doctest, pdb transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/AV_Summaries/001_Pytest_doctest_pdb__9c2a04e725b61a85.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Pytest, doctest, pdb transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/AV_Summaries/001_Pytest_doctest_pdb__9c2a04e725b61a85.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9991; only the JSON segmentation supplies the displayed timestamps.
Pytest, doctest, pdb transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/AV_Summaries/001_Pytest_doctest_pdb__9c2a04e725b61a85.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Pytest, doctest, pdb transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/AV_Summaries/001_Pytest_doctest_pdb__b3f1d7a13157fe7e.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Pytest, doctest, pdb transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/AV_Summaries/001_Pytest_doctest_pdb__b3f1d7a13157fe7e.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.

Source	Path	Use
Pytest, doctest, pdb transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/AV_Summaries/001_Pytest_doctest_pdb__b3f1d7a13157fe7e.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9982; only the JSON segmentation supplies the displayed timestamps.
Pytest, doctest, pdb transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/AV_Summaries/001_Pytest_doctest_pdb__b3f1d7a13157fe7e.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Python Classes and Objects transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/AV_Summaries/008_Python_Classes_and_Objects__67559daa97bde626.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Python Classes and Objects transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/AV_Summaries/008_Python_Classes_and_Objects__67559daa97bde626.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Python Classes and Objects transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/AV_Summaries/008_Python_Classes_and_Objects__67559daa97bde626.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9994; only the JSON segmentation supplies the displayed timestamps.
Python Classes and Objects transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/AV_Summaries/008_Python_Classes_and_Objects__67559daa97bde626.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.

Source	Path	Use
Inheritance transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/AV_Summaries/010_Inheritance__a6ea033571f65071.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Inheritance transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/AV_Summaries/010_Inheritance__a6ea033571f65071.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Inheritance transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/AV_Summaries/010_Inheritance__a6ea033571f65071.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9997; only the JSON segmentation supplies the displayed timestamps.
Inheritance transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/AV_Summaries/010_Inheritance__a6ea033571f65071.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Exceptions transcript (JSON)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/AV_Summaries/015_Exceptions__524460b511d1f7c5.transcript.json	Full JSON sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Exceptions transcript (Markdown)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/AV_Summaries/015_Exceptions__524460b511d1f7c5.transcript.md	Full Markdown sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Exceptions transcript (plain text)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/AV_Summaries/015_Exceptions__524460b511d1f7c5.transcript.txt	Full plain text sidecar. Normalized text similarity to canonical JSON: 0.9999; only the JSON segmentation supplies the displayed timestamps.

Source	Path	Use
Exceptions transcript (SRT)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/AV_Summaries/015_Exceptions__524460b511d1f7c5.srt	Full SRT sidecar. Normalized text similarity to canonical JSON: 1.0000; only the JSON segmentation supplies the displayed timestamps.
Slide page text: UE24CS151A PCPS Unit 3 Sindhu Class 17 18 Pytest doctest pdb	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__17__18_Pytest_doctest_pdb__3cc021de2b8d71a0.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.
Alternate extracted slide page text: UE24CS151A PCPS Unit 3 Sindhu Class 17 18 Pytest doctest pdb	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__17__18_Pytest_doctest_pdb__3cc021de2b8d71a0.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.
Page-render source for UE24CS151A PCPS Unit 3 Sindhu Class 17 18 Pytest doctest pdb (converted PDF; original hash b78c970f591c88f9854f3ff5a639483d06dc76a8e2c1a5800c4e488b827de3db)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/001_UE24CS151A_PCPS_Unit_3_Sindhu_Class__17__18_Pytest_doctest_pdb__3cc021de2b8d71a0.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.
Slide page text: Virtual Environments	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/003_Virtual_Environments__ad787f97fa57d161.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.
Alternate extracted slide page text: Virtual Environments	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/003_Virtual_Environments__ad787f97fa57d161.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.

Source	Path	Use
Page-render source for Virtual Environments (converted PDF; original hash 2dcb096e16cfd6f48004073b4b2115bc1fe9ca4aa6ba1ded8e075f9094ba2996)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/003_Virtual_Environments__ad787f97fa57d161.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.
Slide page text: UE24CS151A PCPS Unit 4 SindhuRPai Class 8 9 OOP	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/007_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__8__9_OOP__b4c4c4e7c25b5c27.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.
Alternate extracted slide page text: UE24CS151A PCPS Unit 4 SindhuRPai Class 8 9 OOP	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/007_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__8__9_OOP__b4c4c4e7c25b5c27.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.
Page-render source for UE24CS151A PCPS Unit 4 SindhuRPai Class 8 9 OOP (converted PDF; original hash 2965d957c0ddb606e258caa68f6cac39a01c30a87266fe53bf99981a0ca48ebb)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/007_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__8__9_OOP__b4c4c4e7c25b5c27.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.
Slide page text: UE24CS151A PCPS Unit 4 SindhuRPai Class 10 Inheritance	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/010_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__10_Inheritance__4dcdb777b0cfaae4.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.
Alternate extracted slide page text: UE24CS151A PCPS Unit 4 SindhuRPai Class 10 Inheritance	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/010_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__10_Inheritance__4dcdb777b0cfaae4.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.

Source	Path	Use
Page-render source for UE24CS151A PCPS Unit 4 SindhuRPai Class 10 Inheritance (converted PDF; original hash 3cacb6bd7bcc912a6b169e4c3cbea336ae046494f1df0c927a54e282f5c19985)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/010_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__10_Inheritance__4dcdb777b0cfaae4.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.
Slide page text: UE24CS151A PCPS Unit 4 SindhuRPai Class 11 Polymorphism	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/011_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__11_Polymorphism__d914ed6dbb4c52c6.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.
Alternate extracted slide page text: UE24CS151A PCPS Unit 4 SindhuRPai Class 11 Polymorphism	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/011_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__11_Polymorphism__d914ed6dbb4c52c6.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.
Page-render source for UE24CS151A PCPS Unit 4 SindhuRPai Class 11 Polymorphism (converted PDF; original hash b68cc8fed979095224217876dccf47c02e508dddfdea2526dda58ed734b0f77a)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/011_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__11_Polymorphism__d914ed6dbb4c52c6.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.
Slide page text: UE24CS151A PCPS Unit 4 SindhuRPai Class 12 Iterators	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/014_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__12_Iterators__a61be1414e88ae54.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.

Source	Path	Use
Alternate extracted slide page text: UE24CS151A PCPS Unit 4 SindhuRPai Class 12 Iterators	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/014_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__12_Iterators__a61be1414e88ae54.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.
Page-render source for UE24CS151A PCPS Unit 4 SindhuRPai Class 12 Iterators (converted PDF; original hash e5e40e2fa3025f5bc0853935d4d4ae59522fb43ef182f9666f00317f7e9340d1)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/014_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__12_Iterators__a61be1414e88ae54.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.
Slide page text: UE24CS151A PCPS Unit 4 SindhuRPai Class 14 15 Exceptional Handling	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/015_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__14__15_Exceptional_Handling__147207e8e0b93741.slides.md	Complete extracted page text from the bucket's .slides.md sidecar.
Alternate extracted slide page text: UE24CS151A PCPS Unit 4 SindhuRPai Class 14 15 Exceptional Handling	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/015_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__14__15_Exceptional_Handling__147207e8e0b93741.slides.txt	The bucket's alternate .slides.txt text sidecar; content differences are kept as separate page points.
Page-render source for UE24CS151A PCPS Unit 4 SindhuRPai Class 14 15 Exceptional Handling (converted PDF; original hash 55483b0bdaee29f016a9549096737614f28ed3e32a991169622b4b12a20ab12a)	https://huggingface.co/buckets/alvin3y10/data/tree/pesuc/PESU_Materials/Sem-1/UE26CS151A_Python_for_computational_Problem_Solving/Unit_04/Slides/015_UE24CS151A_PCPS_Unit_4_SindhuRPai_Class__14__15_Exceptional_Handling__147207e8e0b93741.pdf	Page inventory and visual source for the rendered original deck; figure pages are derived from this local PDF. The original bucket path/hash are retained in datasets/content-index.json.

Historical question-source excerpts are shown only when a page has a strong text-based unit match. They are source references, not new solutions or exam predictions.

Known gaps, conflicts and corrections

1. No historical archive question page met the unit-match candidate threshold; all subject-level and ambiguous pages remain in datasets/question-archive.json.
2. 1 video-to-slide associations were not promoted to a match. Candidate rankings are listed in the transcript corpus and video-slide-matches.csv; the slide decks appear separately.
3. 10 transcript segments have low ASR confidence; they are flagged with timestamps in the transcript blocks and indexed in the corpus.
4. This is a visibly labelled source-preservation draft. Human page-level semantic and visual review is not complete; no unsupported subject summary or formula correction is claimed.
5. The 19.26 GiB MP4 files remain hosted on Hugging Face; the full video path/hash and transcript coverage are indexed. The full audio/video was not watched end-to-end.
6. Formula candidates are machine-extracted text with source locators; use the adjacent original slide page and the formula index before treating a candidate as an authoritative equation.
7. The study PDF contains edited, topic-organized lecture notes and textual slide-page takeaways. Full-page slide images remain in the source/visual index datasets, not embedded as the lesson. Human content, ASR, formula, and page-level visual review remains pending.

Coverage

Evidence is summarized by source to keep the reading copy legible. Full source-unit IDs, locators and decisions remain in pack.json.

Source	Unit counts	Example decisions
SRC00001	1 covered · 0 excluded · 0 gaps	catalogue: Unit title and index copied from the official units.json catalogue; human content review is pending.

Source	Unit counts	Example decisions
SRC00002	3 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0040. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0041 to s0042. Sidecar variants compare...
SRC00003	3 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0040. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0041 to s0042. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00004	3 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0040. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0041 to s0042. Sidecar variants compare...
SRC00005	3 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0020. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0021 to s0040. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0041 to s0042. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00006	2 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0019. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0020 to s0021. Sidecar variants compare...
SRC00007	2 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0019. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0020 to s0021. Sidecar variants compare...
SRC00008	2 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0019. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0020 to s0021. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00009	2 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0019. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0020 to s0021. Sidecar variants compare...
SRC00010	6 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0021. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0022 to s0041. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0042 to s0060. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00011	6 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0021. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0022 to s0041. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0042 to s0060. Sidecar variants compare...
SRC00012	6 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0021. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0022 to s0041. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0042 to s0060. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00013	6 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0021. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0022 to s0041. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0042 to s0060. Sidecar variants compare...
SRC00014	4 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0021. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0022 to s0041. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0042 to s0060. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00015	4 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0021. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0022 to s0041. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0042 to s0060. Sidecar variants compare...
SRC00016	4 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0021. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0022 to s0041. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0042 to s0060. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00017	4 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0021. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0022 to s0041. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0042 to s0060. Sidecar variants compare...
SRC00018	10 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0021. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0022 to s0042. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0043 to s0063. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00019	10 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0021. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0022 to s0042. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0043 to s0063. Sidecar variants compare...
SRC00020	10 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0021. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0022 to s0042. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0043 to s0063. Sidecar variants compare...

Source	Unit counts	Example decisions
SRC00021	10 covered · 0 excluded · 0 gaps	t000000-000299: Timestamp interval t000000-000299; canonical segment IDs s0001 to s0021. Sidecar variants compare... t000300-000599: Timestamp interval t000300-000599; canonical segment IDs s0022 to s0042. Sidecar variants compare... t000600-000899: Timestamp interval t000600-000899; canonical segment IDs s0043 to s0063. Sidecar variants compare...
SRC00022	0 covered · 0 excluded · 28 gaps	p1: Extracted slide page 1 from... p2: Extracted slide page 2 from... p3: Extracted slide page 3 from...
SRC00024	0 covered · 0 excluded · 28 gaps	p1: Original slide page 1 rendered from... p2: Original slide page 2 rendered from... p3: Original slide page 3 rendered from...
SRC00023	0 covered · 0 excluded · 28 gaps	p1: Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human... p2: Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human... p3: Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human...

Source	Unit counts	Example decisions
SRC00025	18 covered · 0 excluded · 0 gaps	p1: Extracted slide page 1 from... p2: Extracted slide page 2 from... p3: Extracted slide page 3 from...
SRC00027	18 covered · 0 excluded · 0 gaps	p1: Original slide page 1 rendered from... p2: Original slide page 2 rendered from... p3: Original slide page 3 rendered from...
SRC00026	18 covered · 0 excluded · 0 gaps	p1: Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human... p2: Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human... p3: Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human...
SRC00028	31 covered · 0 excluded · 0 gaps	p1: Extracted slide page 1 from... p2: Extracted slide page 2 from... p3: Extracted slide page 3 from...
SRC00030	31 covered · 0 excluded · 0 gaps	p1: Original slide page 1 rendered from... p2: Original slide page 2 rendered from... p3: Original slide page 3 rendered from...

Source	Unit counts	Example decisions
SRC00029	31 covered · 0 excluded · 0 gaps	p1: Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human... p2: Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human... p3: Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human...
SRC00031	0 covered · 0 excluded · 26 gaps	p1: Extracted slide page 1 from... p2: Extracted slide page 2 from... p3: Extracted slide page 3 from...
SRC00033	0 covered · 0 excluded · 26 gaps	p1: Original slide page 1 rendered from... p2: Original slide page 2 rendered from... p3: Original slide page 3 rendered from...
SRC00032	0 covered · 0 excluded · 26 gaps	p1: Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human... p2: Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human... p3: Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human...

Source	Unit counts	Example decisions
SRC00034	10 covered · 0 excluded · 0 gaps	p1: Extracted slide page 1 from... p2: Extracted slide page 2 from... p3: Extracted slide page 3 from...
SRC00036	10 covered · 0 excluded · 0 gaps	p1: Original slide page 1 rendered from... p2: Original slide page 2 rendered from... p3: Original slide page 3 rendered from...
SRC00035	10 covered · 0 excluded · 0 gaps	p1: Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human... p2: Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human... p3: Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human...
SRC00037	9 covered · 0 excluded · 0 gaps	p1: Extracted slide page 1 from... p2: Extracted slide page 2 from... p3: Extracted slide page 3 from...
SRC00039	9 covered · 0 excluded · 0 gaps	p1: Original slide page 1 rendered from... p2: Original slide page 2 rendered from... p3: Original slide page 3 rendered from...

Source	Unit counts	Example decisions
SRC00038	9 covered · 0 excluded · 0 gaps	p1: Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human... p2: Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human... p3: Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human...
SRC00040	0 covered · 0 excluded · 25 gaps	p1: Extracted slide page 1 from... p2: Extracted slide page 2 from... p3: Extracted slide page 3 from...
SRC00042	0 covered · 0 excluded · 25 gaps	p1: Original slide page 1 rendered from... p2: Original slide page 2 rendered from... p3: Original slide page 3 rendered from...
SRC00041	0 covered · 0 excluded · 25 gaps	p1: Alternate extracted slide text page 1; normalized text matches the .slides.md page. Human... p2: Alternate extracted slide text page 2; normalized text matches the .slides.md page. Human... p3: Alternate extracted slide text page 3; normalized text matches the .slides.md page. Human...

C Revision and retrieval practice